Polynomial Input Preconditioning for Universal Time Series Forecasting

Jerry Han

Department of Mathematics
Princeton University

Advisor: Prof. Elad Hazan
Second Reader: Prof. Amit Singer

Spring 2026

Abstract

Universal Sequence Preconditioning (USP) [Marsden and Hazan, 2025] is a framework for online prediction in marginally stable linear dynamical systems (LDS) in which convolving observations with a fixed monic Chebyshev polynomial compresses the effective spectral radius of the hidden dynamics, yielding the first sublinear, hidden-dimension-independent regret bounds for this class. This paper empirically investigates whether the USP idea transfers to a setting of patch-based transformer foundation models for time-series forecasting, which the original paper does not cover. The setting differs from USP in three ways: the forecaster is a transformer trained from scratch on a large time-series corpus, not an online learner; the data may be far richer than a finite-dimensional LDS; and the task is multi-step forecasting rather than single-step prediction. To accommodate these differences we propose **polynomial input preconditioning**. We precondition our input and concatenate it as an auxiliary input *channel* alongside the raw target, sidestepping the catastrophic error accumulation that direct target preconditioning would induce under multi-step rollout. Only the per-patch input projection is widened (+12K parameters, 0.11% of Moirai-2 Small). Empirically, on the GIFT-Eval benchmark (97 configurations, 5 seeds, 485 paired comparisons) our method achieves $\Delta\text{MASE}^* = -2.9\%$ over a from-scratch Moirai-2 Small baseline, winning 329/485 paired comparisons; capacity-matched controls (duplicate channel, zero channel) confirm that the specific polynomial content drives the gain. At 100K training steps the improvement grows to -3.9%, and the gain remains $\sim 5\%$ across context lengths 512–4,000. Cross-benchmark validation on FEV-Bench (-2.9% MASE, 72/100 wins) confirms the finding independently.

Contents

1 Preliminaries **3**
- 1.1 Passive Prediction in Linear Dynamical Systems . . . 3
- 1.2 Stability and Finite Memory . . . 3
- 1.3 Linear Predictors and Online Convex Optimization . . . 4
- 1.4 Spectral Filtering: Compressing Long Memory . . . 5

2 Universal Sequence Preconditioning **6**
- 2.1 Polynomial Preconditioning of LDS Observations . . . 6
- 2.2 Chebyshev Polynomials . . . 6

2.3 Chebyshev Minimax . . . 7
2.4 The USP Algorithm . . . 7
2.5 The USP Regret Bound . . . 8

3 Patch-based Transformers for Forecasting **9**
3.1 Patches and the Per-Patch Projection . . . 10
3.2 The Moirai-2 Small Architecture . . . 10
3.3 Pretraining Objective and Optimization . . . 11
3.4 Training Corpus: LOTSA . . . 11
3.5 Inference and Forecasting . . . 11
3.6 Evaluation: GIFT-Eval . . . 12
3.7 The Polynomial Preconditioning Channel . . . 13

4 Empirical Results **14**
4.1 Experimental Setup . . . 15
4.2 Main Results . . . 15
4.3 Ablation: Polynomial Degree, Stride, Basis, and Dropout . . . 16
4.4 Horizon Analysis: The Role of Channel Dropout . . . 18
4.5 FEV-Bench Cross-Benchmark Validation . . . 18
4.6 Fixed vs. Learned Coefficients . . . 18
4.7 Context-Length Robustness . . . 19
4.8 Extended Training (100K Steps) . . . 19

5 Discussion and Limitations **20**

6 Conclusion **20**

A Detailed Methodology **23**
A.1 Training Protocol . . . 23
A.2 Evaluation Protocol . . . 23
A.3 Preconditioning Channel Implementation . . . 23

B Capacity Controls **24**

C Polynomial Ablations **24**
C.1 Polynomial Degree (Chebyshev, $s = 16$, 10% dropout) . . . 24
C.2 Stride (Chebyshev $d = 4$, 10% dropout) . . . 25
C.3 Basis ($d = 4$, $s = 16$, 10% dropout) . . . 25
C.4 Channel Dropout Rate (Chebyshev $d = 4$, $s = 16$) . . . 25

D Polynomial Coefficients **25**

E Domain and Frequency Analysis **27**

F 100K Extended Training **28**

G Inference-Time Channel-Content Ablation **29**

H FEV-Bench Cross-Benchmark Validation **29**

1 Preliminaries

1.1 Passive Prediction in Linear Dynamical Systems

A partially observed linear time-invariant system has hidden state $x_t \in \mathbb{R}^{d_x}$, observed input $u_t \in \mathbb{R}^{d_u}$, and output $y_t \in \mathbb{R}^{d_y}$:

$$x_{t+1} = Ax_t + Bu_t + w_t, \qquad y_t = Cx_t + v_t, \qquad t \in \mathbb{Z}_{\geq 0}, \tag{1}$$

with initial state $x_0 \in \mathbb{R}^{d_x}$. Here A, B, C are unknown system matrices, while w_t and v_t denote process and observation noise. In time-series forecasting there may be no exogenous input; this is recovered by setting $u_t \equiv 0$ or by treating past observations as the relevant input history.

The prediction problem is passive. Unlike control, the learner does not choose u_t to influence the state. At each time t, the learner observes the past history and predicts $\hat{y}_t$; after observing y_t, it suffers loss $\ell_t(\hat{y}_t, y_t)$ (for example a squared error). For a predictor class Π, regret is

$$\mathrm{Regret}_T(\mathcal{A}, \Pi) \;:=\; \sum_{t=1}^{T} \ell_t(\hat{y}_t, y_t) - \min_{\pi \in \Pi} \sum_{t=1}^{T} \ell_t(\hat{y}_t^{\pi}, y_t). \tag{2}$$

Definition 1.1 (Sublinear regret)**.** An online algorithm $\mathcal{A}$ achieves *sublinear regret* with respect to a comparator class Π if $\mathrm{Regret}_T(\mathcal{A}, \Pi) = o(T)$, or equivalently,

$$\frac{1}{T}\mathrm{Regret}_T(\mathcal{A}, \Pi) \;\xrightarrow{T\to\infty}\; 0.$$

That is, the learner's average per-round loss converges to that of the best predictor in the reference class. This is the sense in which the online-learning results below should be read.

1.2 Stability and Finite Memory

The *spectral radius* of a square matrix is $\rho(A) := \max\{|\lambda| : \lambda \in \mathrm{spec}(A)\}$. For a time-invariant linear system, $\rho(A) < 1$ is equivalent to $A^t \to 0$ as $t \to \infty$. For finite-time learning bounds one usually needs the quantitative version

$$\left\|A^t\right\| \;\leq\; M\rho^t \qquad \text{for all } t \geq 0, \qquad \rho \in (0,1), \tag{3}$$

which says that the influence of the remote past decays geometrically. The following truncation lemma is the precise version of the common statement that strictly stable systems have finite effective memory.

Lemma 1.2 (Stable LDSs are approximated by finite-memory predictors; cf. Hazan and Singh, 2025, Lemma 9.6)**.** *Consider the noiseless input-output LDS $x_{t+1} = Ax_t + Bu_t$, $y_t = Cx_t$. Suppose $\left\|A^t\right\| \leq M\rho^t$, $\|B\| \leq \beta$, $\|C\| \leq \gamma$, $\|u_t\| \leq U$ with $\rho \in (0,1)$. Let π_{A,B,C,x_0} denote the exact open-loop LDS predictor and let $\pi_{A,B,C,h}$ be the length-h truncated predictor $\hat{y}_t^{(h)} = \sum_{i=1}^{h} CA^{i-1}Bu_{t-i}$. Then*

$$\sum_{t=1}^{T} \left\|\hat{y}_t^{\pi_{A,B,C,x_0}} - \hat{y}_t^{(h)}\right\| \;\leq\; T \cdot \frac{M\beta\gamma U}{1-\rho}\rho^h + \sum_{t=1}^{\min\{T,d_x\}} \left\|CA^t x_0\right\|. \tag{4}$$

Proof. Unrolling the state dynamics gives the exact predictor $y_t = CA^t x_0 + \sum_{i=1}^{t} CA^{i-1}Bu_{t-i}$. The truncated predictor uses only the first h terms of the input-driven sum: $\hat{y}_t^{(h)} = \sum_{i=1}^{h} CA^{i-1}Bu_{t-i}$. Their difference splits as

$$y_t - \hat{y}_t^{(h)} \;=\; CA^t x_0 \;+\; \sum_{i=h+1}^{t} CA^{i-1}Bu_{t-i}.$$

For the tail of the input-driven response, the triangle inequality and the stability assumption give

$$\Big\| \sum_{i=h+1}^{t} CA^{i-1}Bu_{t-i} \Big\| \;\le\; \sum_{i=h+1}^{\infty} \gamma M \rho^{i-1} \beta U \;=\; \frac{M\beta\gamma U}{1-\rho}\,\rho^{h}.$$

Summing the initial-state term over $t = 1, \ldots, T$ and the tail bound over T rounds gives the result. □

Thus, after the initial transient, a stable system can be approximated to error ε using memory $h = O(\log(T/\varepsilon)/(1-\rho))$ when ρ is close to one. However, note that the required memory can grow poorly as the stability margin $1-\rho$ shrinks.

When $\rho(A) = 1$ (marginally stable, e.g., pure sinusoids), this geometric truncation no longer gives a finite-memory approximation with rapidly vanishing tail; classical regret bounds degrade, with the dependence on d_x often becoming linear or worse.

1.3 Linear Predictors and Online Convex Optimization

The standard convex comparator class for passive prediction is the class of finite-memory linear predictors:

Definition 1.3 (Finite-memory linear predictors)**.** For memory lengths h, k and norm radius κ, define $\Pi^{L}_{h,k,\kappa}$ to be the set of predictors

$$\hat{y}_t \;=\; \sum_{i=1}^{h} M_{1,i}\, y_{t-i} + \sum_{j=1}^{k} M_{2,j}\, u_{t-j}, \qquad \sum_{i=1}^{h} \|M_{1,i}\| + \sum_{j=1}^{k} \|M_{2,j}\| \le \kappa. \tag{5}$$

This class is computationally attractive because it is convex in the predictor matrices. Online Gradient Descent (OGD) therefore gives the following learning guarantee.

Theorem 1.4 (OGD for finite-memory predictors; cf. Hazan and Singh, 2025, Thm. 6.1 and Cor. 9.7)**.** *Let K be a convex parameter set of diameter D, and suppose the loss gradients over K have norm at most G. OGD with step sizes $\eta_t = D/(G\sqrt{t})$ satisfies*

$$\sum_{t=1}^{T} \ell_t(M_t) - \min_{M \in K} \sum_{t=1}^{T} \ell_t(M) \;\le\; \frac{3}{2} G D \sqrt{T}. \tag{6}$$

In particular, finite-memory linear predictors are learnable with $O(\sqrt{T})$ regret, with constants depending on the memory length, norm radius, and data bounds.

There is also an exact structural reason to study linear predictors. In the noiseless case, every finite-dimensional LDS induces a finite-memory autoregressive predictor.

Theorem 1.5 (Cayley–Hamilton representation; cf. Hazan and Singh, 2025, Thm. 9.4)**.** *Consider the noiseless LDS $x_{t+1} = Ax_t + Bu_t$, $y_t = Cx_t$, where $A \in \mathbb{R}^{d_x \times d_x}$ has characteristic polynomial $p_A(\lambda) = \lambda^{d_x} + a_1 \lambda^{d_x - 1} + \cdots + a_{d_x}$. Then there exist matrices $M_{1,i} \in \mathbb{R}^{d_y \times d_y}$ and $M_{2,j} \in \mathbb{R}^{d_y \times d_u}$, $i, j = 1, \ldots, d_x$, such that for every $t > d_x$,*

$$y_t \;=\; \sum_{i=1}^{d_x} M_{1,i}\, y_{t-i} + \sum_{j=1}^{d_x} M_{2,j}\, u_{t-j}. \tag{7}$$

Concretely, one may take $M_{1,i} = -a_i I_{d_y}$ and $M_{2,j} = C\big(\sum_{i=0}^{j-1} a_i A^{j-1-i}\big)B$ (with $a_0 = 1$).

Proof. Write $d = d_x$. Unrolling the state dynamics for d steps gives

$$x_t \;=\; A^d x_{t-d} + \sum_{j=1}^{d} A^{j-1} B u_{t-j}. \tag{8}$$

For each $i = 1, \ldots, d$, similarly $x_{t-i} = A^{d-i} x_{t-d} + \sum_{j=i+1}^{d} A^{j-i-1} B u_{t-j}$, and therefore $A^{d-i} x_{t-d} = x_{t-i} - \sum_{j=i+1}^{d} A^{j-i-1} B u_{t-j}$. By the Cayley–Hamilton theorem, $A^d + a_1 A^{d-1} + \cdots + a_d I = 0$, so $A^d = -\sum_{i=1}^{d} a_i A^{d-i}$. Substituting into (8) and using the identities for $A^{d-i} x_{t-d}$:

$$x_t \;=\; -\sum_{i=1}^{d} a_i x_{t-i} + \sum_{j=1}^{d} \Big(\sum_{i=0}^{j-1} a_i A^{j-1-i} \Big) B u_{t-j}.$$

Multiplying both sides by C gives $y_t = -\sum_{i=1}^{d} a_i \, y_{t-i} + \sum_{j=1}^{d} C \big(\sum_{i=0}^{j-1} a_i A^{j-1-i} \big) B \, u_{t-j}$, which is the claimed recurrence with $M_{1,i} = -a_i I_{d_y}$ and $M_{2,j} = C \big(\sum_{i=0}^{j-1} a_i A^{j-1-i} \big) B$. □

Theorem 1.5 shows that linear predictors are not just a heuristic, and in the noiseless setting they can exactly represent finite-dimensional LDS, with autoregressive coefficients given precisely by the coefficients of A's characteristic polynomial. The drawback is that the memory length depends on the hidden dimension d_x. Lemma 1.2 removes this dependence for strictly stable systems, but replaces it with dependence on the stability margin $1 - \rho$. Spectral filtering addresses precisely this bottleneck.

1.4 Spectral Filtering: Compressing Long Memory

The Cayley–Hamilton representation (Theorem 1.5) requires memory d_x, the hidden dimension. Spectral filtering, introduced by Hazan et al. [2017], compresses this to $O(\log^2 T)$ parameters, independent of both d_x and the stability margin $1 - \rho$, under the following assumptions:

1. A is symmetric (equivalently, diagonalisable with real eigenvalues $\lambda_i \in [-1, 1]$);
2. $\rho(A) \le 1$ (marginal stability is allowed);
3. bounded inputs $\|u_t\|_2 \le 1$ and bounded system norms $\|B\| \, \|C\| \le C_{\text{domain}}$.

The key idea is to replace the raw-history basis $\{u_{t-1}, u_{t-2}, \ldots\}$ with a compressed spectral basis. Because A is symmetric, the impulse response of the LDS decomposes into scalar geometric decays λ_i^t with $\lambda_i \in [-1, 1]$. Since λ_i^t for negative λ_i alternates in sign, one can handle this via a change of basis and reduce to the case $\alpha \in [0, 1]$ (see Hazan and Singh, 2025, Ch. 11 for details). Consider the family of impulse-response vectors

$$\mu_\alpha \;=\; \begin{bmatrix} 1 & \alpha & \alpha^2 & \cdots & \alpha^{T-1} \end{bmatrix}^\top, \qquad \alpha \in [0, 1], \tag{9}$$

and the Hankel-like matrix that integrates over all such responses:

$$Z_T \;:=\; \int_0^1 \mu_\alpha \mu_\alpha^\top \, d\alpha \in \mathbb{R}^{T \times T}, \qquad (Z_T)_{ij} = \frac{1}{i + j + 1} \quad (i, j = 0, \ldots, T-1). \tag{10}$$

The eigenvalues of Z_T decay exponentially [Hazan and Singh, 2025, Thm. 11.1]:

$$\sigma_k(Z_T) \;\le\; 4\pi^2 \exp\!\left(-\frac{k}{\log T} \right) \qquad (k, T > 1). \tag{11}$$

This rapid decay is the reason spectral filtering is efficient: the top $h = O(\log^2 T/\varepsilon)$ eigenvectors $\phi_1, \ldots, \phi_h$ of Z_T suffice to ε-approximate every impulse response in the family $\{\mu_\alpha : \alpha \in [0,1]\}$, and therefore every scalar component of the LDS's response. The spectral predictor projects the input history onto this basis:

$$\hat{y}_t \;=\; \sum_{i=1}^{h} M_i \,\langle \tilde{u}_t, \phi_i \rangle, \tag{12}$$

where $\tilde{u}_t = (u_{t-1}, u_{t-2}, \ldots, u_1, 0, \ldots, 0)$. The coefficients M_i are learned online via OGD (Theorem 1.4), yielding sublinear regret with $h = O(\log^2 T)$ parameters.

2 Universal Sequence Preconditioning

Marsden and Hazan [2025] introduced a polynomial-preconditioning framework that yields the first sublinear, hidden-dimension-independent regret bounds for online prediction in marginally-stable LDSs. Convolving the observation sequence with the coefficients of a fixed degree-d monic polynomial p is algebraically equivalent to multiplying the hidden dynamics by $p(A)$; choosing p to be the monic Chebyshev polynomial then minimax-optimally compresses the spectral radius the downstream learner faces.

2.1 Polynomial Preconditioning of LDS Observations

Consider an LDS as in (1) with noise terms suppressed for exposition, so that $y_t = CA^t x_0$. Write a monic degree-d polynomial in descending powers as $p(z) = \sum_{i=0}^{d} c_i z^{d-i}$ with $c_0 = 1$. The preconditioned observation is

$$\tilde{y}_t \;:=\; \sum_{i=0}^{d} c_i \, y_{t-i} \;=\; C \cdot p(A) \cdot A^{t-d} x_0, \tag{13}$$

where the second equality follows by substituting $y_{t-i} = CA^{t-i}x_0$ and factoring out A^{t-d}. The identity replaces the implicit dependence of $\tilde{y}_t$ on A^{t-d} by $p(A) \cdot A^{t-d}$. If A's real spectrum lies in $[-\rho, \rho]$, the relevant bound is

$$\rho\big(p(A)\big) \;\le\; \sup_{\lambda \in [-\rho,\rho]} |p(\lambda)| \,. \tag{14}$$

Minimising the right-hand side over monic p of degree d is the classical Chebyshev minimax problem (Theorem 2.1), and its solution $p(\lambda) = \rho^d \tilde{T}_d(\lambda/\rho)$ achieves $\sup_{[-\rho,\rho]} |p| \le \rho^d \cdot 2^{1-d}$. Preconditioning by Chebyshev coefficients therefore compresses the effective spectral radius seen by a downstream predictor by a factor of 2^{1-d} in the worst case over $A \in \{A : \mathrm{spec}(A) \subset [-\rho, \rho]\}$.

2.2 Chebyshev Polynomials

The *Chebyshev polynomials of the first kind*, denoted $T_d(z)$ for $d \in \mathbb{Z}_{\ge 0}$, are defined by the recurrence

$$T_0(z) = 1, \quad T_1(z) = z, \quad T_{d+1}(z) = 2z\, T_d(z) - T_{d-1}(z), \tag{15}$$

or equivalently by the trigonometric identity $T_d(\cos\theta) = \cos(d\theta)$. The first several polynomials are $T_2(z) = 2z^2 - 1$, $T_3(z) = 4z^3 - 3z$, $T_4(z) = 8z^4 - 8z^2 + 1$. The leading coefficient of T_d is 2^{d-1}, so the *monic Chebyshev polynomial* is $\tilde{T}_d(z) = T_d(z)/2^{d-1}$. For $d = 4$,

$$\tilde{T}_4(z) = z^4 - z^2 + \tfrac{1}{8}.$$

Two further facts will be used below. First, the trigonometric definition makes Chebyshev polynomials *equioscillate* on $[-1,1]$: $\tilde{T}_d$ attains its L^∞ norm 2^{1-d} with alternating signs at the $d+1$ extremal points $x_k = \cos(k\pi/d)$, $k = 0, \ldots, d$. Second, the monomial-basis coefficients of $\tilde{T}_d$ have ℓ^1 norm $\left\|\boldsymbol{a}^{(d)}\right\|_1 \le 2^{0.3d}$ for all $d \ge 1$ [Marsden and Hazan, 2025, Lem. 3.2]; this controls the noise gain of the preconditioning polynomial and enters the regret bound of Section 2.5 as the comparator-class diameter.

2.3 Chebyshev Minimax

Theorem 2.1 (Chebyshev minimax; cf. Davis, 1975)**.** *Among all real monic polynomials of degree $d \ge 1$, the monic Chebyshev polynomial $\tilde{T}_d$ uniquely minimises $\sup_{x\in[-1,1]} |q(x)|$:*

$$\tilde{T}_d \;=\; \operatorname*{arg\,min}_{q \text{ monic},\ \deg q = d} \max_{x\in[-1,1]} |q(x)|\,, \qquad \max_{x\in[-1,1]} \left|\tilde{T}_d(x)\right| = 2^{1-d}.$$

Proof sketch. By the trigonometric definition $T_d(\cos\theta) = \cos(d\theta)$, T_d ranges over $[-1,1]$ on $[-1,1]$ with $d+1$ alternating extrema $\tilde{T}_d(x_k) = (-1)^k \cdot 2^{1-d}$ at $x_k = \cos(k\pi/d)$, $k = 0, \ldots, d$. Suppose for contradiction that some monic q of degree d has $\sup_{[-1,1]} |q| < 2^{1-d}$. Then $r := \tilde{T}_d - q$, of degree $\le d-1$, takes the same sign as $\tilde{T}_d$ at each x_k and so alternates in sign across $d+1$ points; the intermediate-value theorem gives at least d zeros, forcing $r \equiv 0$, a contradiction. □

Corollary 2.2 (Worst-case spectral compression)**.** *For diagonalisable A with real spectrum in $[-\rho,\rho]$, the choice $q(\lambda) = \rho^d \tilde{T}_d(\lambda/\rho)$ minimises $\sup_A \rho(q(A))$ over monic polynomials of degree d, attaining*

$$\sup \rho\big(q(A)\big) \;=\; \sup_{\lambda\in[-\rho,\rho]} |q(\lambda)| \;\le\; \rho^d \cdot 2^{1-d}.$$

2.4 The USP Algorithm

Leveraging the Chebyshev minimax property, we can state the USP algorithm of Marsden and Hazan [2025]. The starting point is the three-term decomposition of the LDS output. For a system (1) with inputs $u_1, \ldots, u_T$ and monic preconditioning polynomial $p_n^c(z) = \sum_{i=0}^n c_i z^{n-i}$ with $c_0 = 1$, one obtains

$$y_t \;=\; \underbrace{-\sum_{i=1}^{n} c_i\, y_{t-i}}_{\aleph_0 \text{ (preconditioning)}} \;+\; \underbrace{\sum_{s=0}^{n-1}\sum_{i=0}^{s} c_i\, C A^{s-i} B\, u_{t-s}}_{\aleph_1 \text{ (learnable)}} \;+\; \underbrace{\sum_{s=0}^{t-n-1} C\, p_n^c(A)\, A^s B\, u_{t-n-s}}_{\aleph_2 \text{ (residual error)}}. \tag{16}$$

The term $\aleph_0$ is the universal preconditioning component: it depends only on the coefficients $c_{1:n}$ (not on A, B, or C) and can be computed from observed outputs alone. The term $\aleph_1$ involves a bounded number of parameters that can be learned via convex regression. The critical term $\aleph_2$ contains $p_n^c(A)$; if p_n^c is chosen to have small supremum on the spectrum of A, this error term is small.

Algorithm 1 states the general USP framework. During training, each target sequence is *preconditioned* by convolving with the coefficients $c_{0:n}$, and a prediction algorithm $\mathcal{A}$ is trained on the preconditioned targets. At test time, $\mathcal{A}$ predicts from the original (unpreconditioned) inputs, and the preconditioning is inverted by subtracting $\sum_{i=1}^n c_i\, y_{t-i}$. The preconditioning coefficients are fixed (not learned) and independent of the underlying system.

Algorithm 2 instantiates this framework for the online setting with convex regression as the base learner (corresponding to Algorithm 2 of Marsden and Hazan [2025]; the online version of Algorithm 1

Algorithm 1 General Sequence Preconditioning [Marsden and Hazan, 2025, Alg. 1]

Require: Training data $(u_{1:T}^{1:N}, y_{1:T}^{1:N})$; preconditioning coefficients $c_{0:n}$; prediction algorithm $\mathcal{A}$.

Assert $c_0 = 1$.
— **Training** —
for $i = 1$ **to** N **do**
  $y_{1:T}^{\text{pre},\, i} \leftarrow \text{convolve}(y_{1:T}^{i}, c_{0:n})$ $\{y_t^{\text{pre},\, i} = y_t^i + \sum_{j=1}^{n} c_j\, y_{t-j}^i\}$
end for
Train $\mathcal{A}$ on preconditioned data $(u_{1:T}^{1:N}, y_{1:T}^{\text{pre},\, 1:N})$.
— **Test Time** —
for $t = 1$ **to** T **do**
  Receive u_t.
  Predict $\hat{y}_t \leftarrow \mathcal{A}(u_{1:t}, y_{1:t-1}) - \sum_{i=1}^{n} c_i\, y_{t-i}$.
  Receive y_t.
end for

Algorithm 2 USP for Regression [Marsden and Hazan, 2025, Alg. 2]

Require: Initial parameters Q_0; preconditioning coefficients $c_{0:n}$ from the n-th degree monic Chebyshev polynomial; convex constraint set $\mathcal{K} = \{(Q_0, \ldots, Q_{n-1}) : \|Q_j\| \leq C_{\text{domain}} n\, \|\boldsymbol{c}\|_1\}$.

Assert $c_0 = 1$.
for $t = 1$ **to** T **do**
  Receive input u_t.
  Predict $\hat{y}_t(Q^t) = -\sum_{i=1}^{n} c_i\, y_{t-i} + \sum_{j=0}^{n-1} Q_j^t\, u_{t-j}$.
  Observe y_t; suffer loss $\ell_t(Q^t) = \left\|\hat{y}_t(Q^t) - y_t\right\|_1$.
  Update: $Q^{t+1} \leftarrow \text{proj}_{\mathcal{K}}\big(Q^t - \eta_t \nabla_Q \ell_t(Q^t)\big)$.
end for

is their Algorithm 4). The learnable parameters are matrices $Q_0, \ldots, Q_{n-1}$ that approximate the $\aleph_1$ term; the residual $\aleph_2$ is treated as an unlearnable error. This specific instantiation is the one for which Marsden and Hazan derive formal regret bounds (Section 2.5).

Marsden and Hazan [2025] also introduce a more powerful instantiation that combines preconditioning with spectral filtering (their Algorithm 3). Spectral filtering learns both $\aleph_1$ and $\aleph_2$, yielding a better regret rate of $\tilde{O}(T^{-3/13})$ compared to $\tilde{O}(T^{-2/13})$ for regression alone (Theorems 2.3 and 2.4). In both cases, the preconditioning polynomial shrinks the learnable domain, making the online optimisation problem easier.

2.5 The USP Regret Bound

The central theoretical result of Marsden and Hazan [2025] is that USP with Chebyshev coefficients achieves sublinear, hidden-dimension-independent average loss for online prediction in marginally-stable LDSs. We state the general cumulative-loss bound first (note: this bounds total prediction loss, not regret relative to a comparator), then the Chebyshev specialisation.

Theorem 2.3 (USP cumulative loss; general polynomial [Marsden and Hazan, 2025, Thm. D.1])**.** *Let $\{u_t\}_{t=1}^{T} \subset \mathbb{R}^{d_{\text{in}}}$ with $\|u_t\|_2 \leq 1$, and let $\{y_t\}_{t=1}^{T}$ be the output of an LDS (A, B, C) with $\|B\|\,\|C\|\,\kappa \leq C_{\text{domain}}$, where $\kappa = \|P\|\,\left\|P^{-1}\right\|$ for a diagonalisation $A = PDP^{-1}$. Then Algorithm 2*

with any monic degree-n polynomial p_n^c satisfies

$$\sum_{t=1}^{T} \|\hat{y}_t - y_t\|_1 \;\le\; C_{\text{domain}} \left[\tfrac{3}{2}\, n^2 \sqrt{d_{\text{out}}}\, \|\boldsymbol{c}\|_1 \sqrt{T} \;+\; \max_{\lambda \in \text{spec}(A)} |p_n^c(\lambda)|\, T^2 \right]. \tag{17}$$

Proof outline. The bound is the sum of two terms. The first is the online-gradient-descent (OGD) regret $\frac{3}{2} G D \sqrt{T}$ of Hazan and Singh [2025, Thm. 6.1] applied to the loss sequence $\ell_t(Q) = \|\hat{y}_t(Q) - y_t\|_1$ over the convex set $\mathcal{K}$. The diameter is $D \le 2C_{\text{domain}} n \|\boldsymbol{c}\|_1$ (each $\|Q_j\| \le C_{\text{domain}} n \|\boldsymbol{c}\|_1$ by the constraint set) and the gradient bound is $G \le n\sqrt{d_{\text{out}}}$ (since $\|\nabla_{Q_i} \ell_t\|_F \le \sqrt{d_{\text{out}}}\, \|u_{t-i}\|_2 \le \sqrt{d_{\text{out}}}$). The second term bounds the comparator's loss. Setting $\hat{Q}_s = \sum_{i=0}^{s} c_i C A^{s-i} B$ makes $\hat{Q} \in \mathcal{K}$ (since $C_{\text{domain}} \ge \|C\|\,\|B\|\,\kappa$), and the prediction error at $\hat{Q}$ reduces to the $\aleph_2$ tail: $\left\|\hat{y}_t(\hat{Q}) - y_t\right\|_1 \le \|C\|\,\|B\|\,\kappa \cdot \max_\lambda |p_n^c(\lambda)| \cdot T$. Summing over T rounds gives the T^2 term. □

Theorem 2.4 (USP regression with Chebyshev coefficients [Marsden and Hazan, 2025, Thm. 2.1])**.** *Under the hypotheses of Theorem 2.3, if additionally* $\max_{j \in [d_x]} |\arg(\lambda_j)| \le (64 \log_2(8T^{3/2}/(3\sqrt{d_{\text{out}}})))^{-2}$ *and the coefficients $c_{0:n}$ are those of the n-th monic Chebyshev polynomial, then*

$$\frac{1}{T} \sum_{t=1}^{T} \|\hat{y}_t - y_t\|_1 \;\le\; \frac{9\, C_{\text{domain}} \sqrt{d_{\text{out}}}\, \log_2^2(3T)}{T^{2/13}}.$$

Proof sketch. By Theorem 2.3, it suffices to bound $\|\boldsymbol{c}\|_1$ and $\max_\lambda |p_n^c(\lambda)|$ for the monic Chebyshev polynomial. Lemma 3.2 of Marsden and Hazan [2025] gives $\|\boldsymbol{c}\|_\infty \le 2^{0.3n}$, hence $\|\boldsymbol{c}\|_1 \le (n+1) \cdot 2^{0.3n}$. For the spectral bound, Lemma 3.1 of Marsden and Hazan [2025] shows that $\max_\lambda \left|\tilde{T}_n(\lambda)\right| \le 2^{-(n-2)}$ when each eigenvalue λ satisfies $|\arg(\lambda)| \le 1/(64n^2)$. Substituting into (17) gives cumulative loss $\le C_{\text{domain}} [\frac{3}{2} n^2 \sqrt{d_{\text{out}}}\, (n+1)\, 2^{0.3n} \sqrt{T} + 2^{-(n-2)} T^2]$. Balancing these two terms by setting $n = \frac{10}{13} \log_2\!\left(\frac{8T^{3/2}}{3\sqrt{d_{\text{out}}}}\right) = \frac{15}{13} \log_2 T + O(1)$ yields the $T^{-2/13}$ average-loss rate; the polylogarithmic factors from n are absorbed into the stated bound. □

For comparison, vanilla regression without preconditioning (i.e., setting $n = 0$ in Theorem 2.3 so that $\max_\lambda |p_0^c(\lambda)| = 1$) achieves cumulative regret $O(C_{\text{domain}} \sqrt{d_{\text{out}}}\, T^{5/2})$ [Marsden and Hazan, 2025, p. 5], which is *not* sublinear in T. USP with spectral filtering (Algorithm 3 of Marsden and Hazan [2025]) improves the rate further to $\tilde{O}(T^{-3/13})$ by learning the $\aleph_2$ term as well, at the cost of a more complex spectral basis. Both rates are *independent of the hidden dimension d_x*, the signature contribution of USP. These bounds apply to the online LDS setting of Marsden and Hazan [2025]; our empirical method (Section 3.7) uses the same Chebyshev filter but in a different setting (pretrained transformer, batch training), so the formal guarantees do not directly transfer.

3 Patch-based Transformers for Forecasting

Time series foundation models are large pretrained models that produce zero-shot forecasts for arbitrary unseen series without task-specific fine-tuning. Like language foundation models, they are trained on a large and diverse corpus and transfer the learned temporal patterns to new series at inference time. The dominant architecture is the *patch-based transformer*: the input series is segmented into fixed-length patches, each patch is projected into an embedding vector, and a stack of transformer layers models inter-patch dependencies via self-attention. The output head decodes these representations into a predictive distribution over future time steps. Several recent models follow this paradigm, including PatchTST [Nie et al., 2023], Moirai [Woo et al., 2024], TimesFM [Das

et al., 2024], and Chronos [Ansari et al., 2024], differing primarily in training objective, patch encoding, and output parameterisation.

Our experiments are conducted on *Moirai-2 Small* [Liu et al., 2025], the open-source[1] dense, decoder-only successor to the original Moirai encoder [Woo et al., 2024]. We chose Moirai-2 because it is a small (11.4M parameter), publicly reproducible, state-of-the-art universal forecaster. It is small enough to train from scratch on a single H100 within wall-clock budgets compatible with five-seed ablations, yet competitive with much larger models on the GIFT-Eval benchmark. Our preconditioning method modifies only the input projection of this model.

3.1 Patches and the Per-Patch Projection

A patch-based transformer forecaster [Nie et al., 2023, Woo et al., 2024, Liu et al., 2025] proceeds as follows. Given a univariate history $x_{1:T}$, partition it into non-overlapping patches $\mathbf{p}_i = (x_{(i-1)P+1}, \ldots, x_{iP})$ of length P for $i = 1, \ldots, T/P$. A per-patch projection (in Moirai-2, a residual block of two linear layers with a SiLU nonlinearity) maps each patch to an embedding $\mathbf{e}_i \in \mathbb{R}^{d_{\text{model}}}$. These embeddings are fed to a transformer block stack whose output is decoded into a predictive distribution over future patches via a per-patch projection head. Two features of this architecture are salient for our development:

- *Cross-patch information must be learned via attention.* Before the first self-attention operation, $\mathbf{e}_i$ is a function of $\mathbf{p}_i$ alone; it contains no information about $\mathbf{p}_{i-1}$, $\mathbf{p}_{i+1}$, or more distant patches. Patterns whose natural temporal scale exceeds P time steps (e.g., daily periodicities in 15-minute data, spanning 96 steps or 6 patches at $P = 16$) must be discovered end-to-end by the attention layers above the projection.
- *The per-patch projection is narrow.* For Moirai-2 Small ($P = 16$, $d_{\text{model}} = 384$), the projection's input dimension is $2P = 32$ (target plus observation mask), so it has roughly $32 \cdot 384 \approx 12{,}000$ parameters out of 11.4M total ($\sim 0.1\%$). Adding a third P-dimensional input channel costs ≈ 12K additional parameters ($P \cdot d_{\text{model}} = 6{,}144$ in the first linear layer plus $\approx 6{,}000$ in the residual path); altering downstream layers, in contrast, is much more expensive.

These two observations motivate *input-side* preconditioning: a fixed preconditioning polynomial applied before patching can inject cross-patch information into each $\mathbf{e}_i$ at negligible parameter cost, leaving the rest of the network untouched.

3.2 The Moirai-2 Small Architecture

Moirai-2 Small is a decoder-only transformer with $L = 6$ layers, hidden dimension $d_{\text{model}} = 384$, feed-forward dimension $d_{\text{ff}} = 1024$, $h = 6$ attention heads, and patch size $P = 16$. Each transformer block applies pre-norm self-attention with rotary position embeddings (RoPE) [Su et al., 2024] and a SwiGLU feed-forward network [Shazeer, 2020], with RMSNorm [Zhang and Sennrich, 2019] replacing LayerNorm throughout. A binary attention bias [Liu et al., 2025] discriminates between positions in the observed prefix and positions in the prediction horizon. Inputs are z-scored at the sequence level before patching using a packed-sequence variant of standard z-score scaling. Three features of the architecture are particularly relevant to the rest of this paper:

- *Decoder-only with causal mask.* Unlike the masked-language-model encoder used in Moirai 1.0 [Woo et al., 2024], Moirai-2 attends to past patches only. Predictions for patch i depend on $\mathbf{e}_1, \ldots, \mathbf{e}_i$ but not on later patches.

[1] Available on the HuggingFace Hub as `Salesforce/moirai-2.0-R-small`; the architecture and training recipe follow the description in Liu et al. [2025].

- *Multi-token prediction head.* The output projection emits, for each input patch, predictions for the next $K = 4$ patches at 9 quantile levels $\{0.1, 0.2, \ldots, 0.9\}$, yielding $K \cdot 9 \cdot P = 576$ output values per patch.
- *Per-patch input residual block.* The input projection is a two-layer residual MLP $\text{ResBlock}_{2P \to d_{\text{model}}}$ acting on the concatenation $[\mathbf{p}_i; \mathbf{m}_i]$ of patch and observation mask. This is the only layer that touches raw patch values; it is also the only layer modified by our method (Section 3.7 below).

3.3 Pretraining Objective and Optimization

Each training sample is a packed sequence of patched univariate series. A random *prefix ratio* (default 0.3) determines how much of the sequence is treated as observed context; the remainder is the prediction window. Within the prediction window, the target patches are replaced by a learnable mask token, and the model is trained to predict their values. Additionally, with probability `mask_ratio` $= 0.5$, individual patches inside the prediction window are corrupted to encourage robustness to missing observations.

The training loss is the *packed quantile MAE loss*,

$$\mathcal{L} \;=\; \frac{1}{N \cdot |\mathcal{Q}|} \sum_{n,t,q} \rho_q\Big(y_{n,t} - \hat{y}_{n,t}^{(q)} \Big), \tag{18}$$

where the sum runs over patches in the prediction window of all packed samples, $\mathcal{Q} = \{0.1, \ldots, 0.9\}$ is the set of quantile levels, $\hat{y}_{n,t}^{(q)}$ is the model's q-quantile prediction, and $\rho_q(u) = u(q - \mathbf{1}[u < 0])$ is the standard pinball loss [Koenker and Bassett, 1978]. We use AdamW [Loshchilov and Hutter, 2019] with $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-6}$, weight decay 0.1, peak learning rate $\eta = 10^{-3}$, 1,000-step linear warmup, and cosine annealing to zero. Training is performed in bf16 mixed precision with batch size 256 on a single H100 GPU. Anomaly filtering [Liu et al., 2025] resamples training windows whose normalised values exceed a z-score threshold of 8.0 to suppress extreme outliers in LOTSA. The full set of hyperparameters appears in Appendix A.

3.4 Training Corpus: LOTSA

Moirai-2 is pretrained on the *Large-scale Open Time Series Archive* (LOTSA), introduced by Woo et al. [2024], which aggregates ~ 27 billion observations across 9 broad domains: energy, transport, climate, cloud operations, web, sales, healthcare, nature, and economic/financial. Domains are unbalanced, with a few datasets (e.g., Buildings-900K, ERA5) dominating raw observation counts. Therefore, Moirai-2 uses *variate-proportional weighted sampling*, where for each dataset, the sampling weight is inversely proportional to its raw observation count and proportional to the number of distinct variates, which roughly equalises the contribution of large multivariate datasets and small univariate ones. We use the released LOTSA v1 weighting scheme without modification (the `lotsa_v1_moirai2` configuration in the official codebase). Throughout training, each sample is normalised by its own mean and standard deviation (computed within the observed prefix) before patching.

3.5 Inference and Forecasting

At inference, the model is presented with the full observed history followed by mask tokens for the prediction window. Because the multi-token head emits a K=4-patch forecast for every input patch, predictions for the prediction-window patches are read directly from the corresponding output positions; no autoregressive rollout is required for horizons of up to $K \cdot P = 64$ time steps. For

longer horizons, the predicted patches are appended to the context and the procedure repeats. Each forecast at horizon h produces a vector of 9 quantile estimates at each time step; for our headline metric (MASE), we use the median ($q = 0.5$) as the point forecast.

3.6 Evaluation: GIFT-Eval

We evaluate on GIFT-Eval [Aksu et al., 2024], an open benchmark of 97 dataset $\times$ horizon configurations spanning 24 datasets and 7 frequency classes (sub-hourly through yearly), curated to test the zero-shot generalization claim of universal forecasters. Each configuration specifies a held-out split, prediction horizon, and patch-size convention; we follow the recommended convention of $P = 8$ for quarterly data and $P = 32$ for all other frequencies, with context length 4,000. Note that these evaluation-time patch sizes differ from the architecture's default $P = 16$ (Section 3.2); Moirai-2 supports variable patch sizes at inference. Our preconditioning stride s is always set equal to the evaluation-time patch size P. The remainder of this subsection rigorously defines the metric we report.

Mean Absolute Scaled Error (MASE). For a single test series with point forecast $\hat{y}_t$ and ground-truth y_t over a prediction window of length H, the *Mean Absolute Scaled Error* [Hyndman and Koehler, 2006] is defined as

$$\mathrm{MASE}(\hat{y}, y) \;=\; \frac{\frac{1}{H}\sum_{t=1}^{H} |y_t - \hat{y}_t|}{\frac{1}{N-m}\sum_{t=m+1}^{N} \left|y_t^{\text{train}} - y_{t-m}^{\text{train}}\right|}, \tag{19}$$

where $y_{1:N}^{\text{train}}$ is the in-sample (training) portion of the same series and m is the seasonal period (e.g., $m = 24$ for hourly data with daily seasonality, $m = 1$ for non-seasonal series). The denominator is the in-sample mean absolute error of the *seasonal naive* predictor $\hat{y}_t^{\text{snaive}} = y_{t-m}$, providing a series-specific scaling that makes the metric unitless and comparable across series of different magnitudes. A MASE below 1 means the model beats the seasonal-naive predictor on that series in mean absolute error; above 1 means it does not.

Aggregation across a configuration. A GIFT-Eval "configuration" c is a (dataset, prediction-horizon) pair. The benchmark evaluates the model on a fixed test split of multiple windows from each dataset; let $\mathcal{W}_c$ denote this set of test windows. The configuration-level raw MASE is the mean across windows:

$$\mathrm{MASE}(\text{model}, c) \;=\; \frac{1}{|\mathcal{W}_c|} \sum_{(\hat{y},y)\in\mathcal{W}_c} \mathrm{MASE}(\hat{y}, y). \tag{20}$$

Normalised MASE. Because absolute MASE values vary by orders of magnitude across configurations (some series are easy to forecast, some are not), the GIFT-Eval leaderboard divides each configuration's MASE by that configuration's seasonal-naive MASE before aggregating across configurations:

$$\mathrm{MASE}^*(\text{model}, c) \;=\; \frac{\mathrm{MASE}(\text{model}, c)}{\mathrm{MASE}(\text{seasonal naive}, c)}. \tag{21}$$

A configuration-level MASE* below 1 means the model beats seasonal-naive on that configuration; above 1 means it does not. Note that MASE* is the ratio of the model's MASE to the seasonal-naive MASE, both computed using (19); the seasonal-naive denominator therefore cancels, and $\mathrm{MASE}^*(\text{model}, c)$ is equivalently the ratio of the model's window-mean absolute error to the seasonal-naive predictor's window-mean absolute error on configuration c.

Geometric-mean aggregation. The headline aggregate is the geometric mean of MASE* across all $C = 97$ configurations:

$$\overline{\mathrm{MASE}^*}(\mathrm{model}) \;=\; \left(\prod_{c=1}^{C} \mathrm{MASE}^*(\mathrm{model}, c)\right)^{1/C} \;=\; \exp\left(\frac{1}{C}\sum_{c=1}^{C} \log \mathrm{MASE}^*(\mathrm{model}, c)\right). \tag{22}$$

The geometric mean is preferred over the arithmetic mean because it is symmetric in the log-domain ($\log \overline{\mathrm{MASE}^*}$ is the arithmetic mean of $\log \mathrm{MASE}^*$), so the aggregate cannot be dominated by a single configuration on which one model has an extreme MASE. Equivalently, it averages *relative* performance across configurations.

Reporting convention. We report cross-seed mean and standard deviation of $\overline{\mathrm{MASE}^*}$ together with per-configuration paired-comparison win counts (out of 97 per seed; 485 pooled across the 5 seeds we use). Headline relative changes between methods are reported as $\Delta\% = \big(\overline{\mathrm{MASE}^*}(\mathrm{treatment}) - \overline{\mathrm{MASE}^*}(\mathrm{control})\big)/\overline{\mathrm{MASE}^*}(\mathrm{control}) \cdot 100$, so a negative $\Delta\%$ means the treatment achieves lower MASE (an improvement).

3.7 The Polynomial Preconditioning Channel

In addition to the formal guarantees stated in prior sections, Marsden and Hazan [2025] found empirically that USP with fixed Chebyshev or Legendre coefficients reduces prediction error on synthetic linear, nonlinear, and deep-RNN-generated sequences and on the ETTh1 benchmark. A natural question is whether these techniques transfer to patch-based time series forecasters.

One natural approach might be to precondition the target, train the predictor to emit $\hat{\tilde{y}}_{t+1}$, and invert at inference to recover $\hat{y}_{t+1}$. However, this fails at multi-step horizons because the inversion is a causal recurrence

$$\hat{y}_t \;=\; \hat{\tilde{y}}_t \;-\; \sum_{k=1}^{d} c_k\, \hat{y}_{t-k},$$

in which each recovered value $\hat{y}_t$ depends on previously recovered values $\hat{y}_{t-1}, \ldots, \hat{y}_{t-d}$. Any error in $\hat{y}_{t-k}$ feeds forward into all subsequent inversions, so prediction errors compound geometrically across the rollout rather than staying local to each step.

Instead, we leave the forecast target unchanged and compute the preconditioning residual r_t (Eq. 23 below) as an auxiliary input channel concatenated alongside the raw target and the observation mask. Because the model never operates in preconditioned space at inference, no inversion is needed and error accumulation is avoided.

Specialization to patch-based forecasting. Our setting differs from the LDS online-prediction setting in three ways: (i) the forecaster is a pretrained transformer, not an online learner; (ii) the data-generating process is unknown and not assumed to be a finite-dimensional LDS; (iii) rather than *replacing* the target sequence by its preconditioned version (which would require inverting the convolution at inference and is incompatible with multi-step rollout), we *concatenate* the preconditioned sequence as an auxiliary input channel. Concretely, given the coefficients $c_{0:d}$ of the monic Chebyshev polynomial (with $c_0 = 1$), we apply the causal polynomial $\sum_{i=0}^{d} c_i x_{t-is}$ at stride s to the input series x and feed the model the residual

$$r_t \;=\; \sum_{k=1}^{d} c_k\, x_{t-ks}, \tag{23}$$

alongside the original x. We work with the residual rather than the full polynomial output because the original signal is already provided by the unmodified target channel; the residual captures the *novel* cross-patch information injected by preconditioning. Despite the differences from the original LDS setting, the spectral intuition transfers: a Chebyshev preconditioner shapes the input's spectrum minimax-optimally, and a patch-based transformer facing a signal whose spectrum has been shaped to highlight multi-patch correlations is plausibly better-equipped to capture them.

We now describe the architectural modification that delivers the preconditioning channel of Eq. (23) into the model. Let $x \in \mathbb{R}^T$ be a univariate input series, normalised to zero mean and unit variance over the observed prefix. We form the residual $r_t = \sum_{k=1}^{d} c_k \, x_{t-ks}$, where $c_{1:d}$ are the non-leading coefficients of the monic Chebyshev polynomial $\tilde{T}_d$. Because the polynomial uses only past lags ($k \geq 1$), r_t depends only on $x_{t-s}, x_{t-2s}, \ldots, x_{t-ds}$ and never on future values. During training, positions in the prediction window where the target is masked are zeroed before computing the residual, so no future information leaks into r_t. During long-horizon autoregressive inference, when the model must roll out beyond the K-patch prediction window, the previously predicted values are used in place of ground truth when computing r_t for the extended context; channel dropout during training regularises against the resulting noise (see below). The residual is then partitioned into patches $\mathbf{r}_i$ of length P aligned with the target patches $\mathbf{p}_i$. The per-patch input projection is widened to accept the augmented triple $(\mathbf{p}_i, \mathbf{m}_i, \mathbf{r}_i)$:

$$\mathbf{e}_i = \mathrm{ResBlock}_{3P \to d_{\mathrm{model}}}\big([\mathbf{p}_i; \mathbf{m}_i; \mathbf{r}_i]\big) \in \mathbb{R}^{d_{\mathrm{model}}}, \tag{24}$$

where ResBlock consists of a widened linear layer followed by a SiLU nonlinearity, a second linear layer, and a residual connection. Only the first linear layer widens; the transformer decoder, attention layers, output heads, and normalisation layers are unchanged.

Channel dropout. During training, each patch's preconditioning channel is independently zeroed with probability $p_{\mathrm{drop}} = 0.1$; at inference the full preconditioning signal is always supplied. The primary motivation is to improve long-horizon inference, where the preconditioning signal may be noisy because it is computed from the model's own autoregressive rollouts. Dropout increases robustness to missing or degraded preconditioning signals, preventing catastrophic dependence on the auxiliary channel.

Parameter count. For Moirai-2 Small ($P = 16$, $d_{\mathrm{model}} = 384$), widening the first linear layer from $32 \to 384$ (target + mask) to $48 \to 384$ (target + mask + preconditioning) adds $16 \cdot 384 = 6{,}144$ parameters; the residual path contributes another $\approx 6{,}000$. The total overhead is ≈ 12K parameters, or 0.105% of the 11.4M baseline. No other parameters are added.

Algorithms 3 and 4 describe training and inference respectively. Three points are worth noting. First, because the stride s equals the patch size P, the preconditioning residual $\mathbf{r}_i$ at patch i depends on patches $i-1, i-2, \ldots, i-d$, making the computation a simple lag over patches rather than individual time steps. Second, the Chebyshev coefficients are fixed throughout; the only trainable parameters are the widened input projection and the existing transformer and output-projection weights. Third, during autoregressive rollout (Algorithm 4, line 11), the residual $\mathbf{r}$ for newly predicted patches is computed from the model's own median predictions, which may be noisy; the channel dropout applied during training (Algorithm 3, line 8) regularises against this mismatch.

4 Empirical Results

Algorithm 3 Training with Polynomial Input Preconditioning

Require: Packed patched sequence $\mathbf{p}_{1:N}$ with observation masks $\mathbf{m}_{1:N}$; prefix ratio $\alpha = 0.3$; mask ratio $\beta = 0.5$; Chebyshev coefficients $c_{1:d}$ (fixed); stride $s = P$; channel dropout rate $p_{\text{drop}} = 0.1$; multi-token horizon $K = 4$.

Partition: first $\lfloor \alpha N \rfloor$ patches are context; remaining patches are prediction window.

z-score: compute μ, σ from observed values in the context patches. Set $\tilde{\mathbf{p}}_i \leftarrow (\mathbf{p}_i - \mu)/\sigma$ for all i.

Mask: for each prediction-window patch independently with probability β: zero out $\tilde{\mathbf{p}}_i$ and $\mathbf{m}_i$.

for $i = 1$ **to** N **do** {Preconditioning residual}
  $\mathbf{r}_i \leftarrow \sum_{k=1}^{d} c_k \cdot \tilde{\mathbf{p}}_{i-k}$ {Stride $s{=}P \Rightarrow$ lag k patches}
  if Bernoulli(p_{drop}) **then**
    $\mathbf{r}_i \leftarrow \mathbf{0}$ {Channel dropout}
  end if
end for
for $i = 1$ **to** N **do** {Embed}
  $\mathbf{e}_i \leftarrow \text{ResBlock}_{3P \to d_{\text{model}}}([\tilde{\mathbf{p}}_i; \mathbf{m}_i; \mathbf{r}_i])$
end for
$\mathbf{h}_{1:N} \leftarrow \text{TransformerDecoder}(\mathbf{e}_{1:N})$ {Causal attention}
$\hat{\mathbf{y}}_i^{(q,k)} \leftarrow \text{OutProj}(\mathbf{h}_i)$ for quantiles $q \in \mathcal{Q}$, horizons $k \in \{1, \ldots, K\}$.
$\mathcal{L} \leftarrow \frac{1}{|\mathcal{Q}|} \sum_{q,i,k} \rho_q\big(\tilde{\mathbf{p}}_{i+k} - \hat{\mathbf{y}}_i^{(q,k)}\big)$ over prediction-window positions.
Update all trainable parameters (ResBlock, Transformer, OutProj) via $\nabla\mathcal{L}$.

4.1 Experimental Setup

Our experimental protocol exactly matches Section 3: we train Moirai-2 Small from scratch on LOTSA v1 (with the official Moirai-2 weighting) for 10K optimizer steps (and 100K in Section 4.8) and evaluate on GIFT-Eval at context length 4,000 across all 97 dataset $\times$ horizon configurations. At evaluation time, the patch size P varies by frequency ($P = 8$ for quarterly, $P = 32$ otherwise; see Section 3.6), and the preconditioning stride is always set to $s = P$. When we write $s = 16$ in ablation tables, this refers to experiments using the default $P = 16$ configuration. The four conditions of interest below share identical training data, optimizer, schedule, seeds, and evaluation pipeline; only the content of the auxiliary input channel differs. We run five seeds per condition, $\{0, 1, 2, 7, 42\}$, yielding $5 \times 97 = 485$ paired comparisons per contrast. All headline aggregates use the geometric-mean normalised MASE of Eq. (21), with cross-seed mean and standard deviation reported throughout.

4.2 Main Results

We compare four conditions, all with identical Moirai-2 Small architecture, training data, optimizer, and seeds; only the auxiliary input channel content differs. *d4_dropout* (ours) supplies the Chebyshev residual channel of Eq. (23). The two capacity-matched controls test whether the gain is driven by the polynomial content or merely by the extra parameters in the widened input projection: *Zero* fills the auxiliary channel with zeros (the projection layer is widened identically to d4_dropout but receives a constant input), while *Duplicate* fills it with a copy of the target patch (the projection sees the same information twice). Both controls have the same parameter count as d4_dropout (+12K parameters over Baseline). *Baseline* is unmodified Moirai-2 Small.

Algorithm 4 Inference with Polynomial Input Preconditioning

Require: Observed history $x_{1:T}$; prediction length H; Chebyshev coefficients $c_{1:d}$ (fixed); stride $s = P$; multi-token horizon $K = 4$; quantile levels $\mathcal{Q} = \{0.1, \ldots, 0.9\}$.

Ensure: Point forecasts $\hat{x}_{T+1:T+H}$.

1: **z-score:** compute μ, σ from $x_{1:T}$. Set $\tilde{x}_t \leftarrow (x_t - \mu)/\sigma$ for $t = 1, \ldots, T$.
2: **Patchify context:** $\tilde{\mathbf{p}}_i \leftarrow (\tilde{x}_{(i-1)P+1}, \ldots, \tilde{x}_{iP})$, $\mathbf{m}_i \leftarrow \mathbf{1}$ for $i = 1, \ldots, T/P$.
3: Initialise prediction patches: $\tilde{\mathbf{p}}_{T/P+j} \leftarrow \mathbf{0}$, $\mathbf{m}_{T/P+j} \leftarrow \mathbf{0}$ for $j = 1, \ldots, \lceil H/P \rceil$.
4: **Compute residual:** $\mathbf{r}_i \leftarrow \sum_{k=1}^{d} c_k \cdot \tilde{\mathbf{p}}_{i-k}$ for all patches i. {No dropout at inference}
5: **Embed, decode:** $\mathbf{e}_i \leftarrow \mathrm{ResBlock}([\tilde{\mathbf{p}}_i; \mathbf{m}_i; \mathbf{r}_i])$; $\mathbf{h}_{1:N} \leftarrow \mathrm{TransformerDecoder}(\mathbf{e}_{1:N})$.
6: **Read predictions:** from the last context patch $i^* = T/P$, extract $\hat{\mathbf{y}}_{i^*}^{(0.5,k)}$ for $k = 1, \ldots, K$. These are the median forecasts for the next K patches (KP time steps).
7: **if** $H \leq KP$ **then** {Direct prediction}
8: **Return** $\hat{x}_{T+1:T+H} \leftarrow \sigma \cdot \hat{\mathbf{y}}_{1:H}^{(0.5)} + \mu$.
9: **else** {Autoregressive rollout}
10: Fill predicted patches into context: $\tilde{\mathbf{p}}_{i^*+k} \leftarrow \hat{\mathbf{y}}_{i^*}^{(0.5,k)}$, $\mathbf{m}_{i^*+k} \leftarrow \mathbf{1}$.
11: Recompute $\mathbf{r}$ for newly filled patches using predicted values.
12: Repeat from **Embed, decode** with extended context until H steps are covered.
13: **end if**

Table 1: Main results on GIFT-Eval, 5 seeds, 485 paired comparisons per method. $\Delta\%$ is the geometric-mean MASE change relative to Baseline; negative values mean lower MASE (improvement). Win rates are paired per-configuration comparisons. Zero and Duplicate are capacity-matched controls (identical architecture and parameter count to d4_dropout; only the channel content differs).

Method	**GeoMean MASE**	**$\Delta\%$ vs Baseline**	**Wins / 485**
d4_dropout (Ours)	**0.837**	**−2.89%**	**329**
Zero (capacity control)	0.858	−0.45%	262
Baseline	0.862	—	—
Duplicate (capacity control)	0.865	+0.38%	217

Table 1 shows that d4_dropout improves geometric-mean normalised MASE by 2.89% over Baseline at 10K steps and wins on 329 of the 485 paired per-configuration comparisons. d4_dropout also improves evaluation performance substantially more than Zero and Duplicate, showing that the specific Chebyshev residual content, not the extra parameters, drives the gain. (Note: our from-scratch Baseline uses substantially less training compute than the official Moirai-2.0-R-Small release, so the absolute MASE values are higher than published leaderboard numbers. All relative improvements are internal to our controlled setup.)

4.3 Ablation: Polynomial Degree, Stride, Basis, and Dropout

The ablations in Table 2 align with the theoretical predictions of Section 2 and pin down what is and is not load-bearing about d4_dropout's configuration.

(a) Polynomial degree. We vary the preconditioning degree across $d \in \{2, \ldots, 8\}$, running five seeds each. $d = 4$ achieves the lowest mean MASE, but all degrees are within roughly one seed-to-seed standard deviation. Every $d \geq 2$ improves over Baseline. We use $d = 4$ as a working choice.

Table 2: Polynomial ablations on GIFT-Eval (97 configs, 10K steps). Degree row uses 5 seeds (mean); other rows use seed 0. Best in **bold**.

(a) Degree (Chebyshev, $s = 16$, dropout 10%, 5 seeds, mean MASE)

d	2	3	**4**	5	6	7	8
MASE	0.850	0.842	**0.837**	0.842	0.840	0.847	0.841
Std	0.013	0.005	0.009	0.008	0.011	0.011	0.005

(b) Stride (Chebyshev d=4, dropout 10%, seed 0)

s	4	8	**16**	32
MASE	0.841	0.852	**0.823**	0.848

(c) Basis (d=4, s=16, dropout 10%, seed 0)

Basis	**Cheby**	EMA	Legendre	FiniteDiff
MASE	**0.823**	0.833	0.840	0.857

(d) Channel Dropout (Chebyshev d=4, s=16, seed 0)

p_{drop}	0%	2%	**10%**	20%
MASE	0.838	0.839	**0.823**	0.842

(b) Stride. Stride $s = P = 16$ is optimal, beating $s = 4, 8, 32$. This is the empirical signature of the cross-patch interpretation, as the nonzero polynomial coefficients at lags 32 and 64 align exactly with patch boundaries (two and four patches back), so the preconditioning channel injects cross-patch structure at the natural unit of the architecture. We suspect that strides smaller than P contaminate the residual with within-patch information that the per-patch projection already sees; strides much larger than P over-shoot the receptive scales the model can use.

(c) Basis. We compare four polynomial bases, all at $d = 4$, $s = 16$, 10% dropout. To understand the ordering, we describe each basis and its explicit residual coefficients for $d = 4$:

- **Monic Chebyshev** ($\tilde{T}_4(z) = z^4 - z^2 + 1/8$). The residual is $r_t = -x_{t-2P} + \frac{1}{8}x_{t-4P}$, with only two nonzero terms. The L^∞ norm on $[-1, 1]$ is $\sup_{[-1,1]} \left|\tilde{T}_4\right| = 2^{-3} = 1/8 = 0.125$, the minimax-optimal value (Theorem 2.1).

- **Monic Legendre** ($\tilde{P}_4(z) = z^4 - \frac{6}{7}z^2 + \frac{3}{35}$). Defined via the Bonnet recurrence $(d{+}1)P_{d+1}(x) = (2d{+}1)xP_d(x) - dP_{d-1}(x)$, then dividing by the leading coefficient to obtain the monic form. The residual is $r_t = -\frac{6}{7}x_{t-2P} + \frac{3}{35}x_{t-4P}$, the same sparsity pattern as Chebyshev but different coefficients. Legendre polynomials minimise the L^2 norm $(\int_{-1}^{1} p(x)^2\, dx)^{1/2}$ among monic polynomials, but their L^∞ norm is strictly larger: $\sup_{[-1,1]} \left|\tilde{P}_4\right| = 8/35 \approx 0.229$, a $1.83\times$ gap versus Chebyshev (and $\approx 3.35\times$ in the squared term that enters the spectral-radius bound (14)).

- **Exponential moving average** (EMA). A smoothing operation with geometrically decaying weights $a_k \propto \alpha^k$, truncated to $d = 4$ terms. Not derived from a polynomial family. EMA provides some cross-patch mixing but has no minimax optimality property.

- **Finite difference**. The d-th order difference operator $p(z) = (z - 1)^d$. For $d = 4$: $r_t = -4x_{t-P} + 6x_{t-2P} - 4x_{t-3P} + x_{t-4P}$, the discrete fourth derivative. Its L^∞ norm on $[-1, 1]$ is $\sup_{[-1,1]} \left|(z - 1)^4\right| = 2^4 = 16$, amplifying rather than compressing spectral content. It has no spectral-attenuation guarantee.

The empirical ordering Chebyshev (0.823) > EMA (0.833) > Legendre (0.840) > FiniteDiff

(0.857) directly matches the L^∞-norm prediction of the spectral-radius bound (Eq. (14)): the basis that minimises $\sup_{[-1,1]} |p|$ wins, and the bases that provide no attenuation (finite differences) barely help. The Chebyshev–Legendre gap of ~ 1.7 percentage points is consistent with the $1.83\times$ theoretical sup-norm ratio, and the fact that all polynomial bases except finite differences beat Baseline (0.862) confirms that cross-patch information injection is the load-bearing mechanism; the specific coefficients then determine how much benefit is extracted.

(d) Channel dropout. We find that a 10% dropout rate is optimal.

4.4 Horizon Analysis: The Role of Channel Dropout

To understand why channel dropout helps, we compare two variants that differ only in whether channel dropout is applied during training: d4 (Chebyshev $d = 4$, $s = 16$, *no* dropout) and d4_dropout (identical, but with 10% channel dropout). Both use the same fixed Chebyshev residual $r_t = -x_{t-2P} + \frac{1}{8}x_{t-4P}$; the only difference is whether the channel is randomly zeroed during training. We bin the 97 GIFT-Eval configurations by prediction horizon into short ($H < 480$ time steps), medium ($480 \leq H < 960$), and long ($H \geq 960$).

Table 3: d4 vs. d4_dropout vs. Baseline by prediction-horizon bin (seed 0, 10K steps). Both methods beat Baseline at all horizons, but d4_dropout's advantage over d4 grows with the horizon.

Horizon bin	n	**Baseline**	**d4**	**d4_dropout**	**d4** Δ%	**d4_dropout** Δ%
Short	55	0.840	**0.825**	0.828	−1.8%	−1.4%
Medium	21	0.859	0.830	**0.823**	−3.4%	−4.2%
Long	21	0.924	0.880	**0.874**	−4.7%	−5.3%
All	97	0.862	0.838	**0.837**	−2.8%	−2.9%

Table 3 reveals a clear pattern. On short horizons, d4 and d4_dropout are within 0.4% of each other and both beat Baseline by $\sim 1.5\%$. On medium and long horizons, where the model must perform multi-step autoregressive rollout and the preconditioning channel is computed from the model's own predicted values, d4_dropout opens a 0.7–0.9% gap over d4. This is the behaviour channel dropout is designed to produce: during training, randomly zeroing the channel teaches the model not to rely on it catastrophically, so when the channel's signal degrades at long horizons (because it is computed from noisy autoregressive predictions rather than ground-truth), the model degrades gracefully rather than collapsing. The overall d4_dropout vs. d4 gap (−2.9% vs. −2.8%) is small in aggregate, but the horizon-stratified view shows that dropout's benefit is concentrated where it matters: at the long horizons that dominate practical forecasting difficulty.

4.5 FEV-Bench Cross-Benchmark Validation

To verify that our findings generalize beyond GIFT-Eval, we evaluate on the independent FEV-Bench [Shchur et al., 2025] (100 tasks). Following the same protocol as Section 4.2, we report normalized geomean MASE and FEV-Bench's scaled quantile loss (SQL):

The FEV-Bench improvements (−2.91% MASE, −3.09% SQL, 72/100 wins) are comparable in magnitude to the GIFT-Eval improvement.

4.6 Fixed vs. Learned Coefficients

A natural alternative to fixing p to be the monic Chebyshev polynomial is to drop the polynomial structure entirely and let the model learn its own degree-d coefficients $(c_1, \ldots, c_d)$ jointly with the

Table 4: FEV-Bench results (5-seed mean for d4_dropout, single-seed for Baseline).

Method	Geomean MASE	SQL	Wins/100
d4_dropout (Ours)	$\mathbf{1.2532 \pm 0.0064}$	$\mathbf{1.0236 \pm 0.0053}$	**72**
Baseline	1.2908	1.0562	—
$\Delta\%$ vs Baseline	-2.91%	-3.09%	—

rest of the network. This isolates the question: does the gain come from the *specific* Chebyshev coefficients prescribed by Theorem 2.1, or merely from the existence of *some* cross-patch lookback channel that the model can shape end-to-end? We trained two learned variants matching d4_dropout's architecture exactly except that the coefficients are now four trainable scalars: one variant initialised at the Chebyshev coefficients, the other at zero.

Table 5: Fixed Chebyshev versus learned coefficients (seed 0, $d = 4$, $s = 16$, 10% dropout).

Initialisation	Learned coefficients (c_1, c_2, c_3, c_4)	MASE	$\Delta\%$ vs. Baseline
Chebyshev init	$(-0.026, -1.032, -0.123, +0.007)$	0.8332	-3.85%
Zero init	$(+0.308, +0.177, +0.057, +0.045)$	0.8358	-3.57%
Fixed Chebyshev (d4_dropout)	$(0, -1, 0, 0.125)$ *(not learned)*	**0.8234**	$\mathbf{-4.99\%}$

Three things are worth pulling out of Table 5.

(i) Either learned variant beats Baseline. A learnt preconditioner over multi-patch lookbacks captures cross-patch correlations the un-augmented Moirai-2 input projection misses. This rules out the strong null in which any auxiliary lookback channel hurts, and reinforces the cross-patch-bottleneck story: the auxiliary channel acts on the same temporal scales the architecture cannot reach within a single patch.

(ii) Initialisation determines the basin: Chebyshev-init stays near Chebyshev, zero-init drifts to a smoother solution. The Chebyshev-initialised variant ends up close to its starting point: the dominant coefficient remains at lag $2s = 32$ with weight ≈ -1, indicating that the loss landscape has a basin of attraction around the analytic solution. The zero-initialised variant converges to a qualitatively different solution with smoother, more gradually decaying coefficients.

Two distinct local minima of comparable quality suggests that the basin is empirically broad and various preconditioning polynomials that mix multi-patch information may be roughly as good.

(iii) Fixed slightly outperforms either learned variant. This is the empirical signal that the Chebyshev choice is meaningful as an inductive bias, and makes the model more sample-efficient.

4.7 Context-Length Robustness

Universal forecasters routinely tune context length to a deployment-time compute budget. A useful question is therefore not just whether d4_dropout helps at the maximum supported context, but whether the benefit holds when the model is given strictly less history.

The improvement is approximately context-length-invariant. A $\sim 5\%$ gap between d4_dropout and Baseline is preserved across an $8\times$ range of context lengths.

4.8 Extended Training (100K Steps)

(i) The benefit of polynomial preconditioning grows with the training budget: d4_dropout improves over Baseline by -3.88% at 100K versus -2.89% at 10K. *(ii) The capacity controls fail to improve*:

Table 6: MASE on GIFT-Eval at four context lengths (seed 0).

Context	Baseline	d4_dropout	Δ%	Wins/97
512	0.9432	0.8986	−4.73%	62
1,000	0.9166	0.8706	−5.02%	65
2,000	0.8813	0.8331	−5.47%	71
4,000	0.8666	0.8234	−4.99%	66

Table 7: 100K-step training, 5 seeds.

Method	GeoMean MASE	Δ% vs Baseline
d4_dropout (Ours)	**0.844**	**−3.88%**
Baseline	0.878	—
Duplicate	0.886	+0.86%
Zero	0.894	+1.73%

Duplicate and Zero both degrade relative to Baseline at 100K (+0.86% and +1.73% respectively), confirming that the benefit is driven by the specific Chebyshev content, not the additional parameters.

5 Discussion and Limitations

The main empirical finding is that polynomial input preconditioning consistently improves the performance of a from-scratch Moirai-2 Small model, as demonstrated on both GIFT-Eval and FEV-Bench with capacity-matched controls.

A plausible mechanism is the input-side bottleneck present in the Moirai-2 architecture: the per-patch projection can only see one patch before attention. The residual channel provides information from 2 and 4 patches back, so the first embedding already contains cross-patch structure that would otherwise require attention to discover. The Chebyshev coefficients appear well-suited because they provide the minimax spectral compression demonstrated in USP.

We note several limitations. Our experiments do not provide provable guarantees for applying preconditioning to patch-based transformers. The mathematical background serves as principled motivation for choosing the Chebyshev basis, but the formal bounds of Section 2.5 do not directly transfer to this setting.

Moreover, we demonstrate the benefit only at the small (11.4M) Moirai-2 scale; limited compute prevents equally comprehensive experiments at the larger (46M) base scale. The method was also tested primarily in a univariate setting, whereas multivariate extensions may require different considerations.

6 Conclusion

We presented the mathematical background of online learning in linear dynamical systems and the theory behind Universal Sequence Preconditioning, including an exposition of the $\tilde{O}(T^{-2/13})$ average-loss bound for USP with online convex regression. We then applied these ideas to patch-based universal time series forecasters via a minimal architectural intervention: the preconditioned signal is concatenated as an auxiliary input channel, adding 0.11% parameters. Empirically, on the GIFT-Eval benchmark (−2.9% MASE at 10K steps, growing to −3.9% at 100K; 329/485 paired wins) and independently on FEV-Bench (−2.9% MASE, 72/100 wins), polynomial preconditioning

consistently improves a from-scratch Moirai-2 Small model. Capacity-matched controls confirm that the specific polynomial content, not the extra parameters, drives the gain. We believe similar improvements may be obtained by incorporating USP-derived signals into other deep learning architectures for sequential prediction.

Acknowledgments and Contributions

I am deeply grateful to my advisor, Prof. Elad Hazan, for suggesting the connection between universal sequence preconditioning and patch-based time series foundation models, and for numerous conversations that shaped the direction of this work. I thank Prof. Amit Singer for serving as second reader and for feedback on the mathematical exposition. I also thank my collaborators Alex Kawaja and Benjamin Cole, who completed this project jointly with me. Any errors are my own.

Individual contributions. The experimental design, implementation, and writing presented in this paper are my individual work. The framework of universal sequence preconditioning as applied to online prediction of linear dynamical systems is due to Marsden and Hazan [2025]; the extension of that framework to patch-based transformer forecasting, the specific polynomial design, the empirical methodology, the statistical analysis, and all figures and tables were developed by me. Computational experiments were run on the Princeton Research Computing cluster.

Honor Code Declaration. This paper represents my own work in accordance with University regulations.

signed Jerry Han

References

Taha Aksu, Gerald Woo, Juncheng Liu, Xu Liu, Chenghao Liu, Silvio Savarese, Caiming Xiong, and Doyen Sahoo. GIFT-Eval: A benchmark for general time series forecasting model evaluation. *arXiv preprint arXiv:2410.10393*, 2024.

Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, et al. Chronos: Learning the language of time series. In *ICML*, 2024.

Abhimanyu Das, Weihao Kong, Andrew Leber, Rajat Mathews, and Ruhan Sen. A decoder-only foundation model for time-series forecasting. In *ICML*, 2024.

Philip J. Davis. *Interpolation and Approximation*. Dover, 1975.

Elad Hazan and Karan Singh. *Introduction to Online Control*. Manuscript, version 1.1β, 2025.

Elad Hazan, Karan Singh, and Cyril Zhang. Learning linear dynamical systems via spectral filtering. In *NeurIPS*, 2017.

Rob J. Hyndman and Anne B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 2006.

Roger Koenker and Gilbert Bassett. Regression quantiles. *Econometrica*, 46(1):33–50, 1978.

Chenghao Liu, Taha Aksu, Juncheng Liu, Xu Liu, Hanshu Yan, Quang Pham, Silvio Savarese, Doyen Sahoo, Caiming Xiong, and Junnan Li. Moirai 2.0: When less is more for time series forecasting. *arXiv preprint arXiv:2511.11698*, 2025.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

Annie Marsden and Elad Hazan. Universal sequence preconditioning. *arXiv preprint arXiv:2502.06545*, 2025.

John C. Mason and David C. Handscomb. *Chebyshev Polynomials*. CRC Press, 2003.

Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *ICLR*, 2023.

Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

Oleksandr Shchur, Abdul Fatir Ansari, Caner Turkmen, Lorenzo Stella, Nick Erickson, Pablo Guerron, Michael Bohlke-Schneider, and Yuyang Wang. fev-bench: A realistic benchmark for time series forecasting. *arXiv preprint arXiv:2509.26468*, 2025.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 2024.

Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified training of universal time series forecasting transformers. In *ICML*, 2024.

Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *NeurIPS*, 2019.

A Detailed Methodology

A.1 Training Protocol

All experiments use identical training infrastructure on NVIDIA H100/A100 GPUs.

Parameter	Value
Architecture	Moirai-2 Small (6 layers, $d = 384$, $d_{\text{ff}} = 1024$, $P = 16$)
Total parameters	11.4M (Baseline), 11.4M + 12K (d4_dropout)
Training data	LOTSA v1, Moirai-2 config (weighted, variate-proportional)
Steps (10K runs)	100 epochs $\times$ 100 batches = 10,000 steps
Steps (100K runs)	1,000 epochs $\times$ 100 batches = 100,000 steps
Batch size	256
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-6}$, wd = 0.1)
LR schedule	Cosine annealing, peak $\eta = 10^{-3}$, min $\eta = 0$
Warmup	1,000 steps (linear)
Precision	bf16 mixed precision
Anomaly filtering	z-score threshold = 8.0, variance ratio = 0.0 (disabled)
Random seeds	0, 1, 2, 7, 42 (5 seeds for main results)

A.2 Evaluation Protocol

All evaluations use the GIFT-Eval benchmark [Aksu et al., 2024] with 97 dataset$\times$horizon configurations:

- Context length: 4,000 tokens (maximum supported)
- Patch size: 32 for most frequencies, 8 for quarterly (following Moirai conventions)
- Batch size: 64
- Metric: Normalized MASE (each configuration's MASE divided by the seasonal naive baseline's MASE)
- Aggregation: geometric mean across 97 configurations
- Reported quantities: cross-seed mean and standard deviation, per-seed MASE, and per-configuration win counts (out of 97 per seed; $5 \times 97 = 485$ when pooled across seeds)

A.3 Preconditioning Channel Implementation

The Chebyshev polynomial $T_d(z)$ is expanded in the monomial basis to obtain the coefficients $c_0, \ldots, c_d$. The preconditioned signal for time step t is

$$r_t \;=\; \left(\sum_{k=0}^{d} c_k \, x_{t-ks} \right) - x_t,$$

computed over the full (normalized, zero-mean unit-variance) time series before patching. Unobserved time steps are masked to zero. During training with channel dropout p, each patch's preconditioning channel independently has probability p of being replaced with zeros. At inference, the full preconditioning channel is always provided.

The preconditioning channel is concatenated with the target and mask channels before the input projection. The input projection is the only architectural change relative to Baseline; all downstream layers (transformer decoder, output projection) are identical.

B Capacity Controls

The Zero and Duplicate conditions serve as capacity controls: they have the *same architecture and parameter count* as d4_dropout, differing only in the content of the extra channel.

Table 8: Capacity control experiments at 10K steps (5 seeds, $\eta = 10^{-3}$). Same architecture width, different channel content.

Method	Content	Mean MASE	Std	Wins/485
d4_dropout	Chebyshev d=4, s=16	**0.8368**	0.0083	**329**
Zero	All zeros	0.8578	0.0102	262
Baseline	No extra channel	0.8617	0.0062	—
Duplicate	Copy of input	0.8650	0.0081	217

- **Duplicate channel hurts**: win rate 44.7%, *worse* than the 50% expected by chance.
- **Zero channel is neutral**: win rate 54.0%, tiny effect (−0.45%).
- d4_dropout vs. Duplicate: 336/485 wins (d4_dropout beats Duplicate on every seed).
- d4_dropout vs. Zero: 323/485 wins (d4_dropout beats Zero on every seed).

At 100K steps, both controls actively hurt: Duplicate degrades by 0.9% and Zero by 1.7% relative to Baseline. Full table in Appendix F.

C Polynomial Ablations

All ablations use seed 0 with 10% channel dropout unless otherwise noted; the polynomial-degree sweep is the exception and is reported over 5 seeds each.

C.1 Polynomial Degree (Chebyshev, $s = 16$, 10% dropout)

Degree	Residual lags	MASE (mean ± std, $n = 5$)	Δ% vs. BL
2	1 (at $2P$)	0.8502 ± 0.0125	−1.33%
3	1 (at $2P$)	0.8422 ± 0.0053	−2.27%
4	2 (at $2P$, $4P$)	$\mathbf{0.8368 \pm 0.0092}$	**−2.89%**
5	2 (at $2P$, $4P$)	0.8423 ± 0.0078	−2.25%
6	3 (at $2P$, $4P$, $6P$)	0.8400 ± 0.0105	−2.52%
7	3 (at $2P$, $4P$, $6P$)	0.8470 ± 0.0110	−1.71%
8	4 (at $2P$, $4P$, $6P$, $8P$)	0.8414 ± 0.0050	−2.36%
Baseline	—	0.8617 ± 0.0062	—

Every $d \geq 2$ beats Baseline. The mean curve is shallow: $d = 4$ achieves the lowest geomean MASE but $d \in \{3, 5, 6, 8\}$ are within roughly one seed-to-seed standard deviation of it. The extreme $d = 2$ has only one non-trivial residual lag (at $2P$). For odd d, $\tilde{T}_d$ has the parity property $T_d(-z) = -T_d(z)$, which causes half the residual coefficients to vanish; the resulting lag structure for $d \in \{3, 5, 7\}$ matches that of $d - 1 \in \{2, 4, 6\}$ with different weights (see Appendix D for explicit coefficients).

C.2 Stride (Chebyshev $d = 4$, 10% dropout)

Stride	Lookback	MASE	Δ% vs. BL	Wins/97
16 ($= P$)	32, 64 steps	**0.8234**	**−4.99%**	**66**
4	8, 16 steps	0.8406	−3.00%	57
8	16, 32 steps	0.8518	−1.71%	62
32	64, 128 steps	0.8484	−2.10%	63
8+16 (dual)	16, 32, 64 steps	0.8389	−3.20%	62

Stride $s = 16 = P$ is optimal: the nonzero coefficients align exactly with patch boundaries, providing maximally informative cross-patch lookback.

C.3 Basis ($d = 4$, $s = 16$, 10% dropout)

Basis	MASE	Δ%	Wins vs. BL	Wins vs. Cheby
Chebyshev $\tilde{T}_4^*$	**0.8234**	**−4.99%**	66/97	—
EMA	0.8329	−3.89%	65/97	41/97
Legendre $\tilde{P}_4^*$	0.8400	−3.07%	62/97	30/97
Legendre $\tilde{P}_6^*$	0.8397	−3.10%	64/97	50/97
Finite differences	0.8569	−1.12%	54/97	29/97
Duplicate	0.8583	−0.96%	42/97	27/97

The basis ordering Chebyshev > EMA > Legendre > finite differences confirms the theoretical prediction (Theorem 2.1, Proposition D.1): the family that minimises the L^∞ norm on $[-1, 1]$ outperforms families optimised for other norms.

C.4 Channel Dropout Rate (Chebyshev $d = 4$, $s = 16$)

Dropout	MASE	Δ% vs. BL	Wins/97
0% (d4)†	0.8378	−3.50%	—
2%	0.8388	−3.20%	63
3%	0.8497	−1.95%	55
10% **(d4_dropout)**	**0.8234**	**−4.99%**	**66**
20%	0.8422	−2.81%	67
30%	0.8559	−1.23%	61

†d4 MASE is the 5-seed mean. d4 and d4_dropout are essentially tied; dropout helps marginally.

D Polynomial Coefficients

This appendix lists the explicit polynomial coefficients for every basis discussed in this paper. Throughout, we use the convention of Section 2: writing a monic polynomial in descending powers as $p(z) = \sum_{i=0}^{d} c_i z^{d-i}$ with $c_0 = 1$, the residual applied to the input is $r_t = \sum_{k=1}^{d} c_k \, x_{t-ks}$ (the non-leading coefficients at stride s). The default stride is $s = P = 16$.

Monic Chebyshev (first kind), $\tilde{T}_d(x) = T_d(x)/2^{d-1}$. The classical Chebyshev polynomials satisfy the recurrence $T_0 = 1$, $T_1(x) = x$, $T_{d+1}(x) = 2xT_d(x) - T_{d-1}(x)$. After dividing by the leading coefficient 2^{d-1}, the monic forms are obtained explicitly. The residual coefficients $(c_1, \ldots, c_d)$ at stride s are listed in Table 9.

Table 9: Monic Chebyshev (first kind): polynomial coefficients $\tilde{T}_d(x)$ in the monomial basis (descending powers), and the corresponding residual coefficients $(c_1, \ldots, c_d)$ at stride $s = P = 16$. Effective lookback in time-step units is ks for the k-th nonzero coefficient. Boldface marks the default $d = 4$ polynomial used by `d4_dropout` and `d4`.

d	$\tilde{T}_d(x)$	$(c_1, \ldots, c_d)$	Residual r_t at $s = 16$
2	$x^2 - \frac{1}{2}$	$(0, -\frac{1}{2})$	$-\frac{1}{2}x_{t-32}$
3	$x^3 - \frac{3}{4}x$	$(0, -\frac{3}{4}, 0)$	$-\frac{3}{4}x_{t-32}$
4	$x^4 - x^2 + \frac{1}{8}$	$(\mathbf{0}, \mathbf{-1}, \mathbf{0}, \mathbf{\frac{1}{8}})$	$\mathbf{-x_{t-32} + \frac{1}{8}\, x_{t-64}}$
5	$x^5 - \frac{5}{4}x^3 + \frac{5}{16}x$	$(0, -\frac{5}{4}, 0, \frac{5}{16}, 0)$	$-\frac{5}{4}x_{t-32} + \frac{5}{16}x_{t-64}$
6	$x^6 - \frac{3}{2}x^4 + \frac{9}{16}x^2 - \frac{1}{32}$	$(0, -\frac{3}{2}, 0, \frac{9}{16}, 0, -\frac{1}{32})$	$-\frac{3}{2}x_{t-32} + \frac{9}{16}x_{t-64} - \frac{1}{32}x_{t-96}$
7	$x^7 - \frac{7}{4}x^5 + \frac{14}{16}x^3 - \frac{7}{64}x$	$(0, -\frac{7}{4}, 0, \frac{14}{16}, 0, -\frac{7}{64}, 0)$	$-\frac{7}{4}x_{t-32} + \frac{7}{8}x_{t-64} - \frac{7}{64}x_{t-96}$
8	$x^8 - 2x^6 + \frac{5}{4}x^4 - \frac{1}{4}x^2 + \frac{1}{128}$	$(0, -2, 0, \frac{5}{4}, 0, -\frac{1}{4}, 0, \frac{1}{128})$	$-2x_{t-32} + \frac{5}{4}x_{t-64} - \frac{1}{4}x_{t-96} + \frac{1}{128}x_{t-128}$

For odd d, the monic Chebyshev polynomial $\tilde{T}_d(z)$ has zero coefficients at all even powers of z (since $T_d(-z) = -T_d(z)$ for odd d). Writing $\tilde{T}_d(z) = z^d + c_1 z^{d-1} + \cdots + c_d$ in descending powers, the coefficients c_k with even k (i.e., at odd powers of z) vanish. Since the residual $r_t = \sum_{k=1}^{d} c_k x_{t-ks}$ sums over these coefficients, only the odd-indexed c_k contribute, yielding nonzero terms at even multiples of s only. For example, the $d = 3$ residual at $s = 16$ is supported only at lag $2s = 32$; the $d = 5$ residual at $s = 16$ is supported at lags $2s = 32$ and $4s = 64$.

Monic Chebyshev second kind, $\tilde{U}_d(x) = U_d(x)/2^d$. Defined by $U_0 = 1$, $U_1(x) = 2x$, $U_{d+1}(x) = 2xU_d(x) - U_{d-1}(x)$. The first few monic forms and corresponding residuals at $s = 16$ are:

$$\begin{aligned}
\tilde{U}_2(x) &= x^2 - \tfrac{1}{4}, & r_t &= -\tfrac{1}{4}x_{t-32},\\
\tilde{U}_3(x) &= x^3 - \tfrac{1}{2}x, & r_t &= -\tfrac{1}{2}x_{t-32},\\
\tilde{U}_4(x) &= x^4 - \tfrac{3}{4}x^2 + \tfrac{1}{16}, & r_t &= -\tfrac{3}{4}x_{t-32} + \tfrac{1}{16}x_{t-64}.
\end{aligned}$$

The ℓ_∞ norm of monic $\tilde{U}_d$ on $[-1, 1]$ is $1/2^{d-1}$ [Mason and Handscomb, 2003, Eq. 1.41], twice that of $\tilde{T}_d$, so the spectral attenuation predicted by the equioscillation argument is half as strong as for the first kind.

Monic Legendre, $\tilde{P}_d(x) = P_d(x) \cdot 2^d (d!)^2/(2d)!$. Defined by Bonnet's recurrence $(d+1)P_{d+1}(x) = (2d+1)xP_d(x) - dP_{d-1}(x)$, $P_0 = 1$, $P_1(x) = x$. The leading coefficient of P_d is $(2d)!/(2^d(d!)^2)$, so dividing by it yields the monic form. The first few monic Legendre polynomials and residuals at $s = 16$ are:

$$\begin{aligned}
\tilde{P}_2(x) &= x^2 - \tfrac{1}{3}, & r_t &= -\tfrac{1}{3}x_{t-32},\\
\tilde{P}_3(x) &= x^3 - \tfrac{3}{5}x, & r_t &= -\tfrac{3}{5}x_{t-32},\\
\tilde{P}_4(x) &= x^4 - \tfrac{6}{7}x^2 + \tfrac{3}{35}, & r_t &= -\tfrac{6}{7}x_{t-32} + \tfrac{3}{35}x_{t-64}.
\end{aligned}$$

The ℓ_∞ norm of monic $\tilde{P}_d$ on $[-1, 1]$ is strictly larger than that of $\tilde{T}_d$ for all $d \geq 2$. The exact gap is given by the following short proposition, which restates the Chebyshev minimax theorem (Theorem 2.1) against the Legendre L^2 minimiser.

Proposition D.1 (Sup-norm gap between Chebyshev and Legendre)**.** *For $d \geq 1$,*

$$\frac{\sup_{[-1,1]} \left|\tilde{T}_d\right|}{\sup_{[-1,1]} \left|\tilde{P}_d\right|} = \frac{2\binom{2d}{d}}{4^d} \xrightarrow[d\to\infty]{} \frac{2}{\sqrt{\pi d}}.$$

The ratio equals 1 *at* $d = 1$ *and is strictly less than* 1 *for all* $d \geq 2$*, with values* $1, 3/4, 5/8, 35/64, 63/128, \ldots$ *for* $d = 1, 2, 3, 4, 5, \ldots$ *respectively.*

Proof. $\sup_{[-1,1]} \left|\tilde{T}_d\right| = 2^{1-d}$ from Theorem 2.1. For Legendre: $|P_d(x)| \leq 1$ on $[-1, 1]$ with equality at $x = \pm 1$, so the monic $\tilde{P}_d = P_d/c_d$ with leading coefficient $c_d = (2d)!/(2^d(d!)^2)$ has sup norm $1/c_d = 2^d(d!)^2/(2d)!$. Taking the ratio and applying Stirling's approximation gives $2\binom{2d}{d}/4^d \to 2/\sqrt{\pi d}$. □

For $d = 4$, the monic Chebyshev attenuates a worst-case eigenvalue by $1/8 = 0.125$, while the monic Legendre attenuates by only $8/35 \approx 0.229$, a $1.83\times$ gap (or $\approx 3.35\times$ when squared). This explains the $\sim$ 1.9-percentage-point Chebyshev advantage observed in the basis ablation (Table 2(c)). Legendre minimises the ℓ_2 norm $\left(\int_{-1}^{1} p(x)^2\, dx\right)^{1/2}$ rather than the ℓ_∞ norm; for the LDS spectral-radius bound (Eq. (14)), only the ℓ_∞ optimisation is the relevant one.

Other bases used in ablations.

- **Finite-difference** (degree d): $p(z) = (z - 1)^d$. For $d = 4$ at $s = 16$ this expands to $r_t = -4x_{t-16} + 6x_{t-32} - 4x_{t-48} + x_{t-64}$. Large L^∞ norm on $[-1, 1]$ $(= 2^d)$, no spectral-compression guarantee.
- **Exponential moving average** (EMA, half-life τ): geometrically decaying weights $a_k = (1 - \alpha)\alpha^k$, truncated to the window we use. Not polynomial-based.
- **Learned coefficients** (no fixed basis): the model learns the $d = 4$ residual coefficients $(c_1, c_2, c_3, c_4) \in \mathbb{R}^4$ from a Gaussian initialisation jointly with the rest of the network. The learned solution depends on initialisation (Section 4.6, Table 5).

E Domain and Frequency Analysis

Table 10: Domain analysis (d4_dropout vs. BL, 5 seeds pooled, 485 observations).

Domain	n	$\Delta\%$	**Wins**
Demographics	15	−9.83%	13/15
Transport	55	−6.94%	47/55
Energy	80	−3.97%	56/80
M4 Competition	30	−3.83%	23/30
IT/Cloud	120	−2.23%	81/120
ETT (Industrial)	80	−1.50%	54/80
Retail/Sales	20	−0.85%	14/20
Weather	75	−0.34%	36/75
Healthcare	10	−0.24%	5/10

The benefit is consistent with the mechanism, although we caution that the relevant mechanism has two distinct manifestations: (a) the residual's *lookback lags*, which are $2s = 32$ and $4s = 64$ time

steps (this controls which past samples enter the residual channel directly); and (b) the residual polynomial's *frequency selectivity*, i.e. which periodic components are preserved vs. suppressed (at periods $\infty, 32, 16, \ldots$ time steps). The two are mathematically distinct objects and need not coincide with dataset-level periodicities. For 15-minute data, the 32/64-step lookbacks correspond to 8/16 hours of context, useful for transport and energy demand, which exhibit autocorrelation across that range, while the polynomial's suppression periods of $\infty, 8\,\mathrm{h}, 4\,\mathrm{h}, \ldots$ are a distinct set. For daily weather data, both manifestations point to 32/64-day scales which are too coarse to capture weekly/seasonal patterns. The cross-domain pattern is therefore consistent with both lookback and frequency interpretations, but the data does not let us distinguish their contributions.

Table 11: Frequency analysis (d4_dropout vs. BL, 5 seeds pooled).

Frequency	n	$\Delta\%$	**Wins**
Sub-hourly	180	−3.40%	121/180
Hourly	155	−3.80%	113/155
Daily	75	−0.76%	46/75
Weekly	40	−0.73%	29/40
Monthly	25	−3.41%	13/25
Quarterly	5	−0.24%	3/5
Yearly	5	−5.92%	4/5

Prediction horizon. The benefit increases with prediction length: −1.4% for short horizons to −5.1% for long horizons (more negative = larger MASE reduction). Longer horizons require more temporal structure, which the preconditioning channel provides.

F 100K Extended Training

Table 12: Full 100K training budget with capacity controls (5 seeds). d4_dropout vs. BL: −3.9%; d4_dropout vs. Duplicate: −4.7%; d4_dropout vs. Zero: −5.5%.

	s0	**s1**	**s2**	**s7**	**s42**	**Mean**
Baseline	0.9092	0.8920	0.8633	0.8641	0.8636	0.8784
d4_dropout	**0.8349**	**0.8402**	0.8586	**0.8450**	**0.8430**	**0.8443**
Duplicate	0.8887	0.8740	0.8974	0.8794	0.8903	0.8860
Zero	0.8917	0.8585	0.8750	0.8825	0.9610	0.8937
d4_dropout $\Delta\%$ vs. BL	−8.2%	−5.8%	−0.5%	−2.2%	−2.4%	−3.9%

Two findings stand out. First, the benefit of polynomial preconditioning *grows* with training budget: d4_dropout improves by 3.9% at 100K (vs. 2.9% at 10K). Second, the capacity controls that were near-neutral at 10K now actively hurt: Duplicate degrades by 0.9% and Zero by 1.7%. Baseline shows higher variance across seeds at 100K (seed 0 reaches 0.9092 due to late-training instability), while d4_dropout remains stable (range 0.83–0.86).

G Inference-Time Channel-Content Ablation

We take a single d4_dropout model (trained with Chebyshev preconditioning and 10% channel dropout) and, *without retraining*, replace the preconditioning channel's content at inference time. "No preconditioning channel" means the auxiliary channel dimension is removed entirely and inference uses only the target and mask channels (i.e., the Baseline architecture applied to the d4_dropout-trained weights). All other conditions retain the widened input projection but feed it different auxiliary content.

Table 13: Inference-time content ablation (seed 0, d4_dropout-trained model, no retraining). The ordering confirms the model has learned to exploit specific polynomial structure.

Inference channel content	**MASE**	**vs. Normal**
Normal (Chebyshev residual)	0.8234	—
No preconditioning channel (truncated to $2P$ inputs)	0.8666	+5.2%
Zeros (channel present but constant)	0.9003	+9.3%
Random Gaussian noise	0.9427	+14.5%
Duplicate (copy of target patch)	1.0699	+29.9%

The ordering Normal $\gg$ No-channel $>$ Zeros $>$ Random $>$ Duplicate confirms that the model exploits the specific polynomial structure. Random and duplicate channels are worse than no preconditioning at all, demonstrating that the model does not simply use the extra channel for generic capacity.

H FEV-Bench Cross-Benchmark Validation

To verify that our findings generalize beyond GIFT-Eval, we evaluate on FEV-Bench [Shchur et al., 2025] (100 tasks), an independent time-series forecasting benchmark, and reproduce the polynomial-degree sweep.

Table 14: FEV-Bench polynomial-degree sweep, 5 seeds per degree. $\Delta\%$ relative to the single-seed baseline (1.2908).

Degree	**MASE (mean $\pm$ std)**	**$\Delta\%$ vs. BL**	**SQL (mean $\pm$ std)**
2	1.2780 ± 0.0039	-0.99%	1.0425 ± 0.0032
3	1.2609 ± 0.0173	-2.32%	1.0288 ± 0.0125
$4^{\dagger}$	1.2532 ± 0.0064	-2.92%	1.0236 ± 0.0053
5	1.2639 ± 0.0069	-2.08%	1.0309 ± 0.0063
6	1.2521 ± 0.0068	-3.00%	1.0236 ± 0.0060
7	1.2564 ± 0.0114	-2.66%	1.0263 ± 0.0094
8	$\mathbf{1.2518 \pm 0.0082}$	$\mathbf{-3.02\%}$	$\mathbf{1.0228 \pm 0.0059}$

d4_dropout ($d = 4$, our default; marked †) improves geometric-mean MASE by 2.91% on FEV-Bench (5-seed mean 1.2532 ± 0.0064 vs. Baseline 1.2908) and SQL by 3.09% (1.0236 ± 0.0053 vs. 1.0562). The best-performing degree on FEV-Bench is $d = 8$ (bold), though $d \in \{4, 6, 8\}$ are within one standard deviation of each other. The degree sweep reproduces the qualitative GIFT-Eval pattern: every $d \geq 2$ beats Baseline and the curve is shallow.